



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

شبکه‌های کامپیوتری - تمرین دوم
Computer Networks - HW2

شماره درس
۴۰۴۴۳

تاریخ تحویل
بخش عملی ۲۴ آذر - بخش تئوری ۱ دی

مباحث
فصل سوم - لایه انتقال

استاد درس
دکتر سید امیرمهدی صادق‌زاده

نیم‌سال اول سال تحصیلی ۱۴۰۴-۱۴۰۵

پیش از پاسخ به سوالات، به نکات زیر توجه فرمایید.

- خروجی تمرین شما می‌بایست یک فایل ZIP باشد.
- پاسخ‌های بخش عملی و نظری می‌بایست طبق استاندارد زیر در فایل PDF آورده شده باشد.
- لازم به ذکر است که اگر در سوالات بخش عملی از شما کدی خواسته شود آن را در دایرکتوری‌های مجزا، طبق استاندارد زیر قرار دهید.

```
CN_HW#_STDID.pdf <--|
(DIR) Q# <--|
Report.pdf <--|
Others <--|
```

- اشکالات یا ابهامات خود را از طریق تالار پرسش و پاسخ در نظر گرفته شده برای تمرین مطرح نمایید.
- هر گونه نسخه‌برداری از تمرین‌های دیگران منجر به از دست رفتن نمره تمرین خواهد شد.
- نوشتن تمام یا بخشی از پاسخ به زبانی غیر از زبان فارسی منجر به نمره صفر خواهد شد.
- در مجموع ۱۰ روز تاخیر مجاز خواهید داشت که برای هر تمرین ۳ روز را می‌توانید استفاده کنید.
- در صورت استفاده از هر گونه منبع برای پاسخ به سوالات، ذکر اسم و نشانی دقیق و آدرس دسترسی به صفحه مورد نظر الزامی است.
- گزارش مربوط به هر کدام از سوالات عملی در یک فایل جداگانه نوشته شود.
- بارم‌بندی سوالات به قرار زیر است.

بخش	سوال	بارم
سوالات نظری	سوال ۱	۱۵
	سوال ۲	۱۵
	سوال ۳	۱۰
	سوال ۴	۱۰
	سوال ۵	۱۰
	سوال ۶	۱۰
سوالات عملی	سوال ۷	۳۰
	سوال ۸	۱۰۰
جمع نمرات		۲۰۰

سوالات نظری

۱. (۱۵ نمره) یک اتصال TCP را در نظر بگیرید که در حال حاضر در حالت Congestion Avoidance (AIMD) قرار دارد.

- آخرین ACK مربوط به شماره sequence برابر 101 بوده است.
- اندازه CWND برابر 10 (به واحد packet) است.
- بسته‌های شماره 101 تا 110 به ترتیب در زمان‌های $t = 0, 0.1, \dots, 0.9$ (ثانیه) ارسال شدند.
- بسته شماره 102 تنها در ارسال اول خود **lost** می‌شود.
- RTT برابر ۱ ثانیه است.

جدول‌های زیر را تا زمانی که فرستنده بسته شماره 116 را ارسال کند، پر کنید.

(آ) بدون fast recovery
قوانین:

- On new ACK: $CWND \leftarrow CWND + \frac{1}{[CWND]}$.
- On triple dupACKs: $SSTHRESH = \lfloor \frac{CWND}{2} \rfloor$, then $CWND = SSTHRESH$.

Time (sec)	Receive ACK (due to)	CWND	Transmit Seq # (mark retransmits)
1.0	102 (101)	$10 + \frac{1}{10} = 10.1$	111
1.2	102 (103)	10.1	/
1.3	102 (104)	10.1	/

(ب) با fast recovery
قوانین:

- On triple dupACKs: $SSTHRESH = \lfloor \frac{CWND}{2} \rfloor$, then $CWND = SSTHRESH + 3$,
enter fast recovery
- In fast recovery: on every dupACK: $CWND \leftarrow CWND + 1$.
- On new ACK: exit fast recovery, $CWND = SSTHRESH$.

Time (sec)	Receive ACK (due to)	CWND	Transmit Seq # (mark retransmits)
1.0	102 (101)	$10 + \frac{1}{10} = 10.1$	111
1.2	102 (103)	10.1	/
1.3	102 (104)	10.1	/

۲. (۱۵ نمره) یک فرستنده را در نظر بگیرید که در حال ارسال 1000 بایت داده به یک گیرنده از طریق TCP است.

- فرستنده بسته‌هایی با اندازه 100 بایت ارسال می‌کند.
 - اندازه پنجره (window size) برابر 300 بایت است.
 - ISN (initial sequence number) برابر 99 است (بنابراین D100 اولین بسته ارسال شده است، سپس D200، و به همین ترتیب).
 - به خاطر داشته باشید که TCP از sliding window استفاده می‌کند و در صورت timeout یا (اگر TCP Fast Retransmit فعال باشد) هنگام دریافت سه duplicate ACK بسته حاوی بایت بعدی مورد انتظار را مجدداً ارسال می‌کند. در این سؤال فرض کنید TCP Fast Retransmit فعال نیست.
 - لینک ناپایدار است: بسته‌های D200 و D700 در ارسال اولیه drop می‌شوند.
- (آ) جدول زیر را با تمام بسته‌هایی که فرستنده ارسال می‌کند تا زمانی که گیرنده تمام بسته‌ها را دریافت کرده و فرستنده نیز از این موضوع مطمئن شود، پر کنید. برای سادگی فرض کنید بسته‌ها (دیتا و ACKها) به ترتیب می‌رسند. ممکن است نیازی به پر کردن همه سطرها نداشته باشید.

#	Packet Data Offset	Sent on timeout	Dropped?	Cumulative ACK
1	D100			A200
2	D200		X	
3	D300			A200
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				

(ب) (نمره) اگر RTT لینک ۱۰ میلی‌ثانیه و timeout در ابتدا ۳ ثانیه باشد، زمان کلی لازم برای اینکه گیرنده تمام بسته‌ها را دریافت کند و فرستنده نیز از این موضوع مطمئن شود چقدر است؟ فرض کنید بسته‌ها کوچک هستند (تأخیر ارسال ناچیز است) و زمان پردازش ناچیز است، و مقادیر مورد استفاده در محاسبه Retransmission Timeout (RTO) در طول این پروسه ثابت باقی می‌مانند.

۳. (۱۰ نمره) امیر فکر می‌کند که ارسال ACK برای هر بسته خیلی پرهزینه است و می‌خواهد یک پروتکل لایه انتقال طراحی کند که reliable باشد ولی تعداد ACKها بسیار کم باشد (حداکثر $\log n$ تا، که n تعداد بسته‌هاست). پروتکل او به صورت زیر است:

۱. اگر P_i بسته شماره i باشد، وقتی گیرنده هر یک از بسته‌های مجموعه

$$\{P_{2^i} \mid 1 \leq i \leq \lfloor \log_2 n \rfloor\} \cup \{P_n\}$$

را دریافت کند، برای آن بسته ACK می‌فرستد.

۲. فرستنده بسته‌ها را به صورت پنجره‌هایی به ترتیب $[P_{2^{i-1}}, P_{2^{i-1}+1}, \dots, P_{2^i}]$ می‌فرستد (با شروع $i=0$)، و منتظر ACK آخرین بسته در آن پنجره می‌ماند. اگر ACK دریافت نشود، همان پنجره را مجدداً می‌فرستد تا زمانی که ACK آخرین بسته آن پنجره دریافت شود. سپس i یک واحد افزایش می‌یابد و پنجره بعدی فرستاده می‌شود. به عبارت دیگر، پنجره از ۱ بسته شروع می‌کند و هر بار که آخرین بسته پنجره ACK شد، اندازه پنجره دو برابر می‌شود تا زمانی که ACK P_n شود.

(آ) آیا این پروتکل reliable است؟ با استدلال کافی پاسخ دهید.

(ب) اگر پاسخ شما این است که پروتکل reliable نیست، کمترین تغییر ممکن تا آن را reliable کند چیست؟

(ج) اگر پروتکل reliable شود (یا اگر شما اصلاح بالا را اعمال کنید)، آیا در عمل در پهنای باند صرفه‌جویی می‌شود؟ با استدلال کافی پاسخ دهید.

مریم نیز ایده‌ای مشابه دارد تا تعداد ACKها را نصف کند؛ او قوانین زیر را پیشنهاد می‌کند:

۱. قرار است P_{2^i} بسته زوج i ام باشد. گیرنده تنها زمانی ACK می‌فرستد که هم P_{2^i} و هم $P_{2^{i+1}}$ را دریافت کرده باشد. (ایندکس‌بندی بسته‌ها از ۰ شروع شده است).

۲. فرستنده تا زمانی که همه بسته‌های زوج ACK نشده‌اند به ارسال ادامه می‌دهد.

(د) آیا این پروتکل reliable است؟ با استدلال کافی پاسخ دهید.

(ه) در صورتی که پاسخ شما به قسمت قبل خیر است، با چه تغییری در این پروتکل می‌توان آن را reliable کرد؟

(و) آیا این پروتکل واقعاً تعداد ACKها را کاهش می‌دهد؟

۴. (۱۰ نمره) اگر سرور A یک اتصال TCP را از کلاینت B قبول کند، در طی three-way handshake سرور A شماره اولیه sequence خود را به B می‌فرستد و از B یک acknowledgement دریافت می‌کند. بنابراین یک کلاینت دیگر C می‌تواند به شکل زیر وانمود کند که B است:

• C یک بسته SYN می‌فرستد تا اتصال TCP ای با A باز کند و خود را B معرفی می‌کند.

• A به B یک بسته SYN+ACK می‌فرستد که شامل شماره اولیه sequence است، و منتظر acknowledgement می‌ماند.

• B به این SYN+ACK پاسخی نمی‌دهد (آن را نادیده می‌گیرد).

• C یک acknowledgement به A می‌فرستد و وانمود می‌کند که از طرف B است.

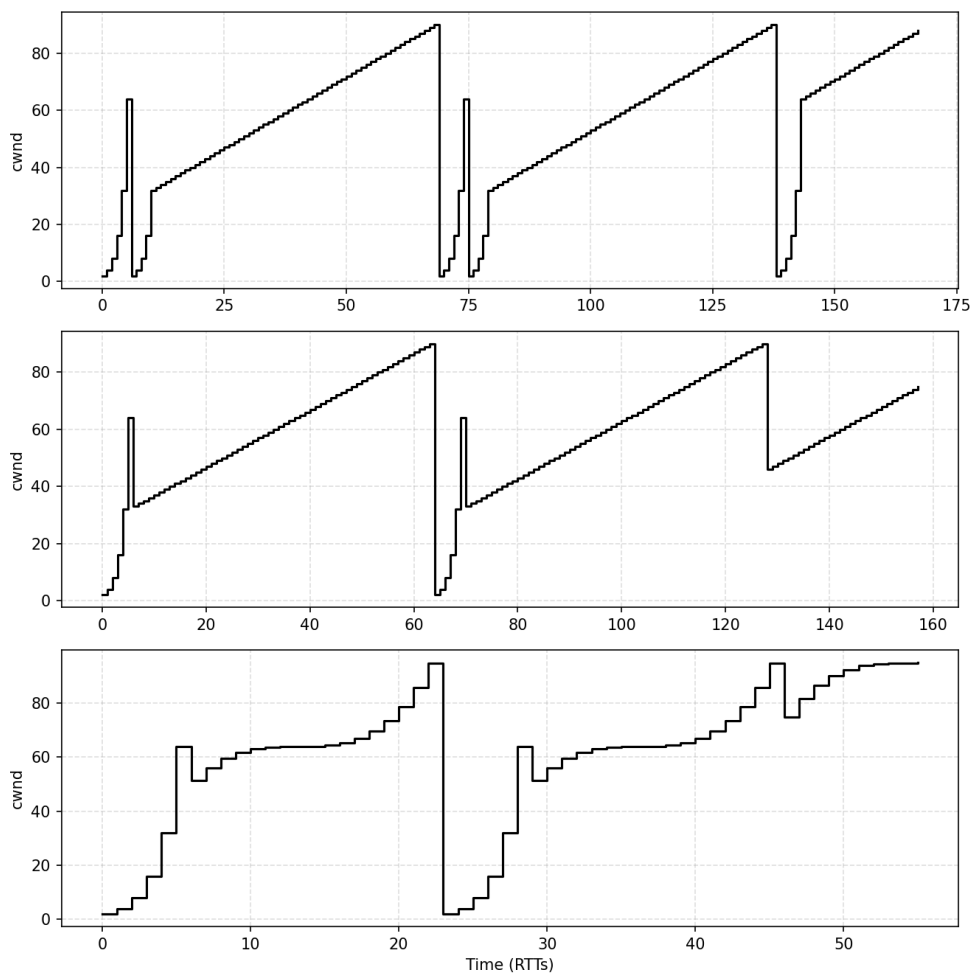
با این حال، استدلال می‌شود که C به درستی نمی‌تواند به A acknowledgement معتبر بفرستد چون شماره اولیه se-quence که A فرستاده را دریافت نکرده است.

الگوریتم انتخاب شماره اولیه sequence بر پایه مقدار یک ساعت در زمان اتصال است. RFC 793 تعیین می‌کند که این مقدار هر $4 \mu s$ یک بار افزایش یابد؛ پیاده‌سازی‌های رایج Berkeley این رفتار را ساده‌سازی کرده و هر ثانیه یک بار مقدار را ۲۵۰۰۰۰ تا افزایش می‌دهند.

(آ) با توجه به پیاده‌سازی ساده شده (افزایش ۲۵۰۰۰۰ در ثانیه)، توضیح دهید که چگونه C می‌تواند (دست‌کم در مرحله باز کردن اتصال) وانمود کند که B است.

(ب) با فرض اینکه RTT واقعی را بتوان تا حدود 40 ms تخمین زد، در پیاده‌سازی غیر ساده شده (افزایش هر $4 \mu s$) به‌طور متوسط انتظار دارید انجام استراتژی قسمت (آ) به چند تلاش نیاز داشته باشد؟

۵. (۱۰ نمره) در تصویر زیر یک نمونه شبیه‌سازی از فرایند کنترل ازدحام در پروتکل TCP را می‌بینید. با توجه به این تصویر به سوالات پاسخ دهید.



(آ) برای هر یک از سه نمودار بالا، مشخص کنید رفتار هر نمودار با کدام نوع از کنترل ازدحام TCP تطابق بیشتری دارد. در مورد هر کدام توضیح دهید.

(ب) با توجه به هر سه نمودار، تمام افت‌های قابل مشاهده در مقدار cwnd و هر تغییر محسوس در رفتار منحنی را شناسایی کرده و برای هر رویداد، نوع آن و واکنش الگوریتم کنترل ازدحام را به‌صورت دقیق توضیح دهید. در مواردی که علت وقوع یک رویداد به‌طور یکتا قابل تشخیص نیست، تمام سناریوهای ممکن را بیان کرده و برای هر سناریو استدلال کنید. در توضیحات خود، ناحیه یا نقطه مورد نظر را با استفاده از اعداد مشخص شده روی شکل ارجاع دهید.

(ج) برای هر یک از سه الگوریتم نمایش داده شده در نمودارها، یک مزیت و یک عیب رفتار پنجره ازدحام آن‌ها را بیان کنید.

(د) کدام الگوریتم پس از وقوع رویداد از دست رفتن بسته، تهاجمی‌ترین رفتار را در افزایش مجدد مقدار cwnd نشان می‌دهد؟ این ویژگی چه تأثیری بر گذردهی^۱ خواهد داشت؟

۶. (۱۰ نمره) دو قطعه UDP زیر روی سیم شنود شده‌اند. در هر دو مورد، بایت اولی که می‌بینید، اولین بایت سرآیند UDP است (هیچ سرآیند IP ای نشان داده نشده است).

Segment A: C0 01 00 35 00 20 00 00 ...

Segment B: 00 35 C0 01 00 40 9F 10 ...

فرض کنید هر دو سگمنت از قالب استاندارد UDP استفاده می‌کنند؛ یعنی چهار فیلد ۱۶ بیتی big-endian به ترتیب زیر در سرآیند قرار دارند:

[Checksum][Length][port Destination][port Source]

(آ) برای سگمنت A و B، مقادیر زیر را به دست آورید:

۱. شماره پورت مبدأ

۲. شماره پورت مقصد

۳. طول کل دیتاگرام UDP

۴. طول داده UDP

(ب) می‌دانیم پورت 53 برای DNS رزرو شده است و پورت‌های ≥ 49152 به عنوان پورت‌های موقتی (ephemeral) سمت کلاینت استفاده می‌شوند.

۱. با استناد به شماره پورت‌ها و طول‌ها، کدام سگمنت به احتمال زیاد DNS request و کدام DNS response است؟ استدلال خود را توضیح دهید.

۲. کدام جهت (کلاینت به سرور یا سرور به کلاینت) حامل پیام کاربردی بزرگ‌تر است و این اختلاف چند بایت است؟

(ج) در سگمنت A مقدار فیلد checksum عمداً برابر 0000 قرار داده شده، در حالی که در سگمنت B مقدار آن نامساوی صفر است.

۱. دقیقاً توضیح دهید مقدار 0000 برای UDP checksum از دید گیرنده چه معنایی دارد.

۲. فرض کنید برای سگمنت B آدرس آبی مبدأ برابر 10.0.0.1 و آدرس آبی مقصد برابر 192.168.213.125 مشاهده شده است و payload این سگمنت فقط از صفر تشکیل شده است. با در نظر گرفتن pseudo-header در IPv4، توضیح دهید چگونه می‌توان بررسی کرد که آیا مقدار checksum درج شده در هدر (0x9F10) با این آدرس‌ها و طول دیتاگرام سازگار است یا نه. عبارت جمع واژه‌های ۱۶ بیتی‌ای را که باید در محاسبه شرکت داده شوند برای این سگمنت به طور کامل بنویسید و در نهایت نتیجه بگیرید checksum معتبر است یا خیر.

سوالات عملی

۷. (۳۰ نمره) در این تمرین، هدف بررسی پروتکل TCP و مشکلات احتمالی در لایه انتقال شبکه است. شما با چالش‌ها و مسائلی که ممکن است در این لایه به وجود آید، آشنا خواهید شد و تأثیرات آن‌ها را شبیه‌سازی و تحلیل خواهید کرد. این تمرین شامل چهار بخش می‌باشد، که در هر بخش یک سرور و یک کلاینت در نظر گرفته شده است.

راه‌اندازی محیط

در این تمرین شما برای راه‌اندازی محیط تمرین به ابزار Docker احتیاج خواهید داشت. برای نصب Docker می‌توانید از این [لینک](#) استفاده کنید. در فایل پیوست این تمرین، ساختاری مشابه ساختار زیر را خواهید دید:

```
cn-hw2-practical.zip
├── normal
│   └── docker-compose.yaml
├── part-a
│   └── docker-compose.yaml
├── part-b
│   └── docker-compose.yaml
├── part-c
│   └── docker-compose.yaml
└── part-d
    └── docker-compose.yaml
```

- پوشه **normal**: شامل نسخه کامل و بدون خطا از ساختار کلاینت-سرور است. این پوشه می‌تواند به عنوان مرجع برای بررسی نحوه اجرای صحیح سرویس‌ها مورد استفاده قرار گیرد.
- پوشه‌های **part-a** تا **part-d**: این پوشه‌ها شامل فایل‌های docker-compose.yaml مربوط به بخش‌های مختلف تمرین هستند. هر پوشه یک سناریوی خاص را برای تحلیل، عیب‌یابی و بررسی ارتباط بین کلاینت و سرور فراهم می‌کند.

برای اجرا کردن هر محیط کافی است به پوشه مربوطه بروید و دستور زیر را در آن محیط اجرا کنید.

```
1 docker compose up -d
```

این دستور ایمیج‌های مربوط به بخش مورد نظر را pull کرده و اجرا می‌کند. از آن‌جا که ممکن است به هر دلیلی در هنگام دریافت ایمیج‌ها از مخزن داکر خطایی رخ دهد، فایل‌های این ایمیج‌ها برای هر دو معماری amd64 و arm64 در اختیار شما قرار گرفته‌اند.

برای استفاده از این فایل‌ها، کافی است نسخه متناسب با معماری سیستم خود را دانلود کرده و اسکریپت `load-images.sh` موجود در آن را اجرا کنید تا ایمیج‌ها به صورت محلی در محیط داکر شما load شوند.

بعد از اجرای هر محیط می‌توانید وضعیت اجرای محیط را با استفاده از دستور `docker compose ps` مشاهده کنید. اگر به هر دلیلی نیاز داشتید که محیط به حالت اولیه خود برگردد و از اول راه‌اندازی شود، می‌توانید دستورهای زیر را اجرا کنید.

```
1 docker compose down
2 docker compose up -d
```

برای دسترسی به ترمینال سرور

```
1 docker compose exec server bash
```

و برای دسترسی به ترمینال کلاینت از

```
1 docker compose exec client bash
```

استفاده کنید.

توجه کنید برای اجرای صحیح دستورهای `docker compose` باید در پوشه مربوط به بخش مورد نظر باشید تا خروجی‌های مربوط به آن را ببینید.

در سرور تمامی بخش‌ها یک `http server` ساده در حال اجرا می‌باشد که می‌توانید از داخل ترمینال کلاینت با دستور زیر به آن درخواست `http` ارسال کنید.

```
1 curl -v http://server
```

بخش‌ها

در هر چهار بخش این تمرین، اختلالی در برقراری اتصال عادی بین کلاینت و سرور ایجاد شده است. در هر محیط، ابزارهای متداول عیب‌یابی شبکه مانند `curl`، `nc`، `telnet`، `tcpdump` و... در دسترس شما قرار دارد و هدف این است که با استفاده از این ابزارها، به منشأ مشکل پی ببرید و اثرات آن را شرح دهید.

همچنین، در صورتی که نیاز به `capture` کردن پکت‌های هر بخش با استفاده از `Wireshark` داشتید، می‌توانید از پروژه [Edgeshark](#) استفاده کنید. توجه کنید که حل این سؤال‌ها بدون استفاده از این ابزار نیز امکان‌پذیر است و نیازی به نصب آن وجود ندارد، اما می‌توانید برای دید عمیق‌تر نسبت به ساختار درونی شبکه مورد استفاده در `Docker` از آن بهره بگیرید.

بخش a تا c

در این بخش‌ها، در صورتی که تلاش کنید از کلاینت به سرور درخواست ارسال کنید، درخواست شما با مشکل مواجه خواهد شد و ارتباط با سرور به درستی برقرار نمی‌شود. شما باید با استفاده از شواهد کافی، مرحله‌ای از اتصال که در آن مشکل به وجود می‌آید را شناسایی و تحلیل کنید.

بخش d

این بخش برخلاف سه بخش قبلی، پاسخ درخواست کلاینت به درستی ارسال می‌شود، اما این پاسخ با تأخیر قابل مشاهده‌ای نسبت به حالت عادی ارسال خواهد شد. در این بخش، شما باید ارتباط فعلی را با حالت عادی مقایسه کنید، پکت‌های درخواست را بررسی کرده و تفاوت‌ها را شرح دهید.

پاسخ

پاسخ شما برای این تمرین باید شامل شواهد کافی از مشکل ایجاد شده باشد. هدف از سوالات، شناسایی نحوه ایجاد مشکل نمی‌باشد و تمرکز بر تفسیر مشکل و یادگیری استفاده از ابزارهای معمول عیب‌یابی شبکه است. یکی از نکات کلیدی در این سوالات، تعیین محل وجود مشکل بین سرور و کلاینت می‌باشد، که باید با استفاده از شواهد و آزمایش‌های مناسب این موضوع نیز شناسایی شود.

در صورتی که علت مشکل ایجاد شده بدون شواهد کافی از اثر آن بیان شود و یا از روش‌های غیرمعمول یا استفاده از ابزارهایی خارج از ابزارهای استاندارد در محیط‌های مربوطه برای استخراج اطلاعات استفاده شده باشد، نمره‌ای به آن تعلق نخواهد گرفت.

۸. (۱۰۰ نمره) در این تمرین، با پیاده‌سازی یک سرور و کلاینت که سرور هم به درخواست‌های DNS پاسخ می‌دهد و هم می‌تواند درخواست‌های کلاینت را نیز پروکسی کند با مفاهیم Tunneling و DNS بیشتر آشنا خواهید شد. در واقع در این تمرین یک سرور و کلاینت برای DNS Tunneling پیاده خواهید کرد. به طور خلاصه، کلاینت و سرور شما به عنوان دو سر یک تانل عمل می‌کنند که داده‌های اصلی را در قالب بسته‌های DNS برای یکدیگر ارسال می‌کنند. به عنوان خروجی، باید یک کد client.py و یک کد server.py داشته باشید که دو سر تانل را پیاده‌سازی می‌کنند.

● موارد درخواستی:

(آ) (۱۵ نمره) کلاینت باید روی پورت 3128 یک پروکسی http ایجاد کند. برای ارسال ترافیک به این پورت می‌توان از دستور curl -x http://localhost:3128 ifconfig.io استفاده کرد. این دستور درخواست‌ها را به جای ارسال به سرور اصلی، برای سرور پروکسی ارسال می‌کند.

(ب) (۲۰ نمره) کد سرور شما باید روی پورت 5454 به درخواست‌های DNS گوش کند. سرور شما باید بتواند درخواست‌های DNS عادی را از درخواست‌هایی که از طرف سمت دیگر تانل برایش ارسال شده تفکیک کند. درخواست‌های عادی باید با استفاده از یک Forwarder مشخص شده مثل 4.2.2.4 یا 8.8.8.8، درخواست DNS را پاسخ‌دهند. اما درخواست‌هایی که از سمت تانل ارسال شده‌اند باید به گونه‌ی دیگری مدیریت شوند. برای بررسی عملکرد سرور خود به عنوان یک DNS Server، می‌توانید از دستوراتی مثل nslookup -port=5454 ifconfig.io 127.0.0.1 استفاده کنید. نحوه‌ی تفکیک درخواست‌های عادی و بسته‌های مربوط به تانل بر عهده‌ی شماست و از هر روشی می‌توانید استفاده کنید. اما توجه کنید که سرور DNS شما باید عملکرد اصلی خود یعنی پاسخ به انواع درخواست‌های DNS را داشته باشد.

(ج) (۳۰ نمره) بعد از تشخیص این که یک بسته مربوط به تانل است، سرور باید با پارس کردن Payload، درخواست را به مقصد اصلی آن ارسال کند و بعد از دریافت پاسخ، آن را از طریق تانل (داخل یک بسته‌ی DNS و در قالب پاسخ درخواست ارسالی از سوی کاربر) به کلاینت ارسال کند.

(د) (۲۰ نمره) اندازه‌ی Payload در بسته‌های DNS محدودیت دارد. توجه کنید که در صورت بزرگ بودن داده‌ای که می‌خواهید از کلاینت تانل خود ارسال کنید، باید بسته شکسته شده و در چند بسته‌ی DNS ارسال شود.

(ه) (۵ نمره) دو سر تانل شما، باید امکان مدیریت چند درخواست همزمان را داشته باشد.

(و) (۱۰ نمره) در کلاینت این تانل، می‌توان به جای HTTP Proxy، از Socks Proxy هم استفاده کرد. با کلاینت خود، یک پروکسی Socks روی پورت 8080 ایجاد کنید که مشابه پورت HTTP می‌تواند ترافیک را دریافت کرده و از روی تانل عبور دهد.

● نکات مهم:

— ارتباط بین کلاینت و سرور را قابل اطمینان فرض کنید و این‌گونه در نظر بگیرید که بسته‌های شما بدون مشکل به مقصد خواهند رسید.

— از این پروکسی فقط برای درخواست‌های از نوع HTTP/1.1 و بدون Persistency استفاده خواهد شد.

— کلاینت و سرور شما فقط باید درخواست‌های HTTP را پاسخ دهد. درخواست‌های HTTPS در این تمرین مدنظر نیست.